Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №4

з дисципліни
«Об’єктно-орієнтоване програмування»

на тему

«Вдосконалення структури коду графічного

редактора об’єктів»

Виконав:

студент групи ІМ-52
Пращук Данило Павлович
Номер в списку - 16

Перевірив:

Рекечинський Д. О.

Київ 2026

Завдання

1. Створити у середовищі розробки проект з ім’ям Lab4.

2. Написати вихідний текст програми згідно варіанту завдання.

3. Скомпілювати вихідний текст і отримати виконуваний файл програми.

4. Перевірити роботу програми. Налагодити програму.

5. Проаналізувати та прокоментувати результати та вихідний текст програми.

6. Оформити звіт.

Варіанти завдань та основні вимоги

1. Номер варіанту завдання дорівнює номеру зі списку студентів у журналі.

Студенти з непарним номером (1, 3, 5, . . .) програмують глобальний статичний об'єкт класу MyEditor.

Студенти з парним номером (2, 4, 6, . . .) програмують динамічний обєкт класу MyEditor, забезпечивши коректне його створення та знищення.

3. Усі кольори та стилі (за винятком "гумового" сліду) геометричних форм – як у попередній лабораторній роботі №3. "Гумовий" слід при вводі усіх фігур малювати пунктирною лінією.

4. Окрім чотирьох типів фігур, які були у попередніх лаб. №2 та 3, запрограмувати ще введення та відображення двох нових фігур – лінія з кружечками та каркас куба.

Кольори ліній та заповнення цих нових фігур студент визначає на свій розсуд.

Для об’єктів типів лінії з кружечками та каркасу кубу відповідні класи Запрограмувати саме множинним успадкуванням.

Втім, якщо мова програмування не дозволяє множинне успадкування, то

його можна замінити іншою конструкцією (наприклад, композицією).

У першу чергу це стосується метода Show для нових фігур – для

відображення ліній треба використовувати виклики метода Show з класу LineShape, для відображення кружечків – виклики метода Show з класу EllipseShape, а для відображення прямокутників – виклики метода Show з класу RectShape.

5. Для усіх шести типів форм зробити кнопки Toolbar з підказками (tooltips).

6. У звіті повинна бути схема успадкування класів – діаграма класів. Для побудови діаграми класів дозволяється обрати інструменти на власний розсуд.

main.cpp

```
#include <QApplication>
#include <QMainWindow>
#include "mainWindow.h"

int main(int argc, char *argv[])
{
   QApplication app(argc, argv);
   MainWindow window;



   window.show();
   return app.exec();
}
```

mainWindow.cpp

```
#include "mainWindow.h"
#include <QMenu>
#include <QMenuBar>
#include <QToolBar>
#include <QIcon>
#include <QActionGroup>
#include <QMessageBox>

MainWindow::MainWindow(QWidget *parent) : QMainWindow(parent)
{
   setWindowTitle("Lab 4");
   resize(640, 480);

   editor = new MyEditor(this);
   setCentralWidget(editor);


   QMenuBar *menu = menuBar();
   QMenu *menuFile = menu->addMenu("&Файл");
   QAction *fileAction = menuFile->addAction("Очистити");

   toolBar = addToolBar("Панель фігур");
   toolBar->setMovable(false);


   QActionGroup *shapeGroup = new QActionGroup(this);
   shapeGroup->setExclusive(true);
   QAction *toolbarPoint = new QAction(QIcon(":icons/point.png"), "Крапка", this);
   toolbarPoint->setCheckable(true);
   toolbarPoint->setToolTip("Інструмент: Крапка");
   toolbarPoint->setData(ShapePoint);
```

```
QAction *toolbarLine = new QAction(QIcon(":icons/line.png"), "Лінія", this);
toolbarLine->setCheckable(true);
toolbarLine->setToolTip("Інструмент: Лінія");
toolbarLine->setData(ShapeLine);

QAction *toolbarRect = new QAction(QIcon(":icons/rectangle.png"), "Прямокутник", this);
toolbarRect->setCheckable(true);
toolbarRect->setToolTip("Інструмент: Прямокутник (від центру)");
toolbarRect->setData(ShapeRect);

QAction *toolbarEllipse = new QAction(QIcon(":icons/ellipse.png"), "Еліпс", this);
toolbarEllipse->setCheckable(true);
toolbarEllipse->setToolTip("Інструмент: Еліпс");
toolbarEllipse->setData(ShapeEllipse);

QAction *toolbarLineDots = new QAction(QIcon(":icons/linedots.png"), "Лінія з крапками", this);
toolbarLineDots->setCheckable(true);
toolbarLineDots->setToolTip("Інструмент: Лінія з крапками");
toolbarLineDots->setData(ShapeLineDots);


QAction *toolbarCube = new QAction(QIcon(":icons/cube.png"), "Каркас куба", this);
toolbarCube->setCheckable(true);
toolbarCube->setToolTip("Інструмент: Каркас куба");
toolbarCube->setData(ShapeCube);


shapeGroup->addAction(toolbarPoint);
shapeGroup->addAction(toolbarLine);
shapeGroup->addAction(toolbarRect);
shapeGroup->addAction(toolbarEllipse);
shapeGroup->addAction(toolbarLineDots);
shapeGroup->addAction(toolbarCube);

toolBar->addAction(toolbarPoint);
toolBar->addAction(toolbarLine);
toolBar->addAction(toolbarRect);
toolBar->addAction(toolbarEllipse);
toolBar->addAction(toolbarLineDots);
toolBar->addAction(toolbarCube);

QMenu *menuObjects = menu->addMenu("&Об'єкти");
menuObjects->addAction(toolbarPoint);
menuObjects->addAction(toolbarLine);
menuObjects->addAction(toolbarRect);
menuObjects->addAction(toolbarEllipse);
menuObjects->addAction(toolbarLineDots);
menuObjects->addAction(toolbarCube);
```

```
   QMenu *menuInfo = menu->addMenu("&Довідка");
   QAction *infoAction = menuInfo->addAction("Довідкова інформація");

   connect(fileAction, &QAction::triggered, editor, &MyEditor::clearCanvas);

   connect(infoAction, &QAction::triggered, this, [this]()
        { QMessageBox::information(this, "Довідка",
   "Лабораторна робота №4 «Вдосконалення структури коду графічного редактора об’єктів».<br>"
                            "Варіант: 16<br>"
                            "Автор: Пращук Данило Павлович ІМ-52"); });

   connect(shapeGroup, &QActionGroup::triggered, this, &MainWindow::onNotify);

   toolbarPoint->setChecked(true);
   onNotify(toolbarPoint);
}

MainWindow::~MainWindow()
{
   delete editor;
}

void MainWindow::onNotify(QAction *action)
{

   int shapeType = action->data().toInt();
   switch (shapeType)
   {
      case ShapePoint:
         editor->setCurrentShape(new PointShape());
         setWindowTitle("Lab 4 - Крапка");
         break;

      case ShapeLine:
         editor->setCurrentShape(new LineShape());
         setWindowTitle("Lab 4 - Лінія");
         break;

      case ShapeRect:
      editor->setCurrentShape(new RectShape());
      setWindowTitle("Lab 4 - Прямокутник");
      break;

      case ShapeEllipse:
      editor->setCurrentShape(new EllipseShape());
      setWindowTitle("Lab 4 - Еліпс");
```

```
        break;

        case ShapeLineDots:
        editor->setCurrentShape(new LineDotsShape());
        setWindowTitle("Lab 4 - Лінія з крапками");
        break;

        case ShapeCube:
        editor->setCurrentShape(new CubeShape());
        setWindowTitle("Lab 4 - Каркас куба");
        break;

        default:
           break;
    }
}
```

mainWindow.h

```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>
#include "myEditor.h"


class QToolBar;

class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    MainWindow(QWidget *parent = nullptr);
    ~MainWindow();


private slots:
    void onNotify(QAction *action);
private:
MyEditor *editor;
QToolBar *toolBar;

};
```

```
#endif
```

shape.cpp

```
#include "shape.h"
#include <cmath>
#include <QPen>
#include <QColor>

Shape::Shape() : xs1(0), ys1(0), xs2(0), ys2(0) {}

void Shape::set(long x1, long y1, long x2, long y2)
{
   xs1 = x1;
   ys1 = y1;
   xs2 = x2;
   ys2 = y2;
}

void Shape::rubber(QPainter &painter)
{
   painter.setPen(QPen(Qt::red, 1, Qt::DashLine));
   painter.setBrush(Qt::NoBrush);
}

void PointShape::show(QPainter &painter)
{
   painter.setPen(QPen(Qt::black, 6, Qt::SolidLine, Qt::RoundCap));
   painter.drawPoint(xs1, ys1);
}
Shape *PointShape::clone() const { return new PointShape(*this); }


void LineShape::show(QPainter &painter)
{
   painter.setPen(QPen(Qt::black, 2, Qt::SolidLine));
   painter.drawLine(xs1, ys1, xs2, ys2);
}
void LineShape::rubber(QPainter &painter)
{
   painter.setPen(QPen(Qt::red, 1, Qt::DashLine));
   painter.drawLine(xs1, ys1, xs2, ys2);
}
Shape *LineShape::clone() const { return new LineShape(*this); }


void RectShape::show(QPainter &painter)
```

```
{
   painter.setPen(QPen(Qt::black, 2, Qt::SolidLine));
   painter.setBrush(getBrush());
   //changes
   int dx = std::abs(xs2 - xs1);
   int dy = std::abs(ys2 - ys1);
   QRect previewRect(xs1 - dx, ys1 - dy, 2 * dx, 2 * dy);
   painter.drawRect(previewRect.normalized());
}
void RectShape::rubber(QPainter &painter)
{
   painter.setPen(QPen(Qt::red, 1, Qt::DashLine));
   painter.setBrush(Qt::NoBrush);

   //changes
   int dx = std::abs(xs2 - xs1);
   int dy = std::abs(ys2 - ys1);
   QRect previewRect(xs1 - dx, ys1 - dy, 2 * dx, 2 * dy);

   painter.drawRect(previewRect.normalized());
}
Shape *RectShape::clone() const { return new RectShape(*this); }

void EllipseShape::show(QPainter &painter)
{
   painter.setPen(QPen(Qt::black, 2, Qt::SolidLine));
   painter.setBrush(Qt::NoBrush);

   QRect previewRect(QPoint(xs1,ys1), QPoint(xs2,ys2));
   painter.drawEllipse(previewRect.normalized());
}
void EllipseShape::rubber(QPainter &painter)
{
   painter.setPen(QPen(Qt::red, 1, Qt::DashLine));
   painter.setBrush(Qt::NoBrush);
   QRect previewRect(QPoint(xs1,ys1), QPoint(xs2,ys2));
   painter.drawEllipse(previewRect.normalized());
}
Shape *EllipseShape::clone() const { return new EllipseShape(*this); }


LineDotsShape::LineDotsShape() : LineShape(), EllipseShape() {}
LineDotsShape::LineDotsShape(const LineDotsShape &other): Shape(other), LineShape(other),
EllipseShape(other) {}

void LineDotsShape::set(long x1, long y1, long x2, long y2) {
   LineShape::set(x1, y1, x2, y2);
   EllipseShape::set(x1, y1, x2, y2);
```

```
}

void LineDotsShape::show(QPainter &painter) {
   LineShape::show(painter);
   int savedX1 = xs1, savedY1 = ys1, savedX2 = xs2, savedY2 = ys2;


   int rx = 6;
   int ry = 6;
   EllipseShape::set(savedX1 -rx, savedY1 - ry, savedX1 + rx, savedY1 + ry);
   EllipseShape::show(painter);

   EllipseShape::set(savedX2 - rx, savedY2 - ry, savedX2 + rx, savedY2 + ry);
   EllipseShape::show(painter);

   LineShape::set(savedX1, savedY1, savedX2, savedY2);
}

void LineDotsShape::rubber(QPainter &painter) {
   LineShape::rubber(painter);

   long savedX1 = xs1, savedY1 = ys1, savedX2 = xs2, savedY2 = ys2;

   int rx = 6;
   int ry = 6;
   EllipseShape::set(savedX1 -rx, savedY1 - ry, savedX1 + rx, savedY1 + ry);
   EllipseShape::show(painter);

   EllipseShape::set(savedX2 - rx, savedY2 - ry, savedX2 + rx, savedY2 + ry);
   EllipseShape::show(painter);

   LineShape::set(savedX1, savedY1, savedX2, savedY2);
}

Shape *LineDotsShape::clone() const { return new LineDotsShape(*this); }



CubeShape::CubeShape() : RectShape(), LineShape() {}
CubeShape::CubeShape(const CubeShape &other) : Shape(other), RectShape(other), LineShape(other) {}

void CubeShape::set(long x1, long y1, long x2, long y2) {
   RectShape::set(x1, y1, x2, y2);
   LineShape::set(x1, y1, x2, y2);
}

void CubeShape::show(QPainter &painter) {
```

```
    long origX1 = xs1, origY1 = ys1, origX2 = xs2, origY2 = ys2;

    int dx = std::abs(origX2 - origX1);
    int dy = std::abs(origY2 - origY1);

    long fx1 = origX1 - dx;
    long fy1 = origY1 - dy;
    long fx2 = origX1 + dx;
    long fy2 = origY1 + dy;

    int shiftX = dx / 2;
    int shiftY = dy / 2;

    long bx1 = fx1 + shiftX;
    long by1 = fy1 + shiftY;
    long bx2 = fx2 + shiftX;
    long by2 = fy2 + shiftY;

    RectShape::set(origX1, origY1, origX2, origY2);
    RectShape::show(painter);

    long bCenterX = (bx1 + bx2) / 2;
    long bCenterY = (by1 + by2) / 2;
    long bRadiusX = std::abs(bx2 - bx1) / 2;
    long bRadiusY = std::abs(by2 - by1) / 2;
    RectShape::set(bCenterX, bCenterY, bCenterX + bRadiusX, bCenterY + bRadiusY);
    RectShape::show(painter);

    LineShape::set(fx1, fy1, bx1, by1);
    LineShape::show(painter);

    LineShape::set(fx2, fy1, bx2, by1);
    LineShape::show(painter);

    LineShape::set(fx1, fy2, bx1, by2);
    LineShape::show(painter);

    LineShape::set(fx2, fy2, bx2, by2);
    LineShape::show(painter);

    RectShape::set(origX1, origY1, origX2, origY2);
    LineShape::set(origX1, origY1, origX2, origY2);
}

void CubeShape::rubber(QPainter &painter) {
    long origX1 = xs1, origY1 = ys1, origX2 = xs2, origY2 = ys2;

    int dx = std::abs(origX2 - origX1);
```

```
    int dy = std::abs(origY2 - origY1);

    long fx1 = origX1 - dx;
    long fy1 = origY1 - dy;
    long fx2 = origX1 + dx;
    long fy2 = origY1 + dy;

    int shiftX = dx / 2;
    int shiftY = dy / 2;

    long bx1 = fx1 + shiftX;
    long by1 = fy1 + shiftY;
    long bx2 = fx2 + shiftX;
    long by2 = fy2 + shiftY;

    RectShape::set(origX1, origY1, origX2, origY2);
    RectShape::rubber(painter);

    long bCenterX = (bx1 + bx2) / 2;
    long bCenterY = (by1 + by2) / 2;
    long bRadiusX = std::abs(bx2 - bx1) / 2;
    long bRadiusY = std::abs(by2 - by1) / 2;
    RectShape::set(bCenterX, bCenterY, bCenterX + bRadiusX, bCenterY + bRadiusY);
    RectShape::rubber(painter);

    LineShape::set(fx1, fy1, bx1, by1);
    LineShape::rubber(painter);

    LineShape::set(fx2, fy1, bx2, by1);
    LineShape::rubber(painter);

    LineShape::set(fx1, fy2, bx1, by2);
    LineShape::rubber(painter);

    LineShape::set(fx2, fy2, bx2, by2);
    LineShape::rubber(painter);

    RectShape::set(origX1, origY1, origX2, origY2);
    LineShape::set(origX1, origY1, origX2, origY2);
}

Shape *CubeShape::clone() const { return new CubeShape(*this); }
```

shape.h

```
#ifndef SHAPE_H
```

```
#define SHAPE_H

#include <QPainter>
#include <QRect>
#include <QPoint>

enum ShapeType {
    ShapePoint,
    ShapeLine,
    ShapeRect,
    ShapeEllipse,
    ShapeLineDots,
    ShapeCube
};

class Shape
{
protected:
    long xs1, ys1, xs2, ys2;

public:
    Shape();
    virtual ~Shape() = default;

     virtual void set(long x1, long y1, long x2, long y2);
    virtual void show(QPainter &painter) = 0;
    virtual void rubber(QPainter &painter);
```

```
    virtual Shape* clone() const = 0;
};

class PointShape : public Shape
{
public:
    void show(QPainter &painter) override;
    Shape* clone() const override;
};

class LineShape : virtual public Shape
{
public:
    void show(QPainter &painter) override;
    void rubber(QPainter &painter) override;
    Shape* clone() const override;
};

class RectShape : virtual public Shape
{
public:
    void show(QPainter &painter) override;
    void rubber(QPainter &painter) override;
    Shape* clone() const override;
    virtual QBrush getBrush() {
        return QColor(128,128,128);
    }
```

```
};

class EllipseShape : virtual public Shape
{
public:
    void show(QPainter &painter) override;
    void rubber(QPainter &painter) override;
    Shape* clone() const override;
};

class LineDotsShape : public LineShape, public EllipseShape {
    public:

    LineDotsShape();
    LineDotsShape(const LineDotsShape &other);
        void set(long x1, long y1, long x2, long y2) override;
        void show(QPainter &painter) override;
        void rubber(QPainter &painter);
        Shape* clone() const override;

};

class CubeShape : public LineShape, public RectShape {
    public:
        CubeShape();
    CubeShape(const CubeShape &other);
        void set(long x1, long y1, long x2, long y2) override;
```

```
    void show(QPainter &painter) override;
    void rubber(QPainter &painter);
    Shape* clone() const override;
    QBrush getBrush() override { return Qt::NoBrush; }
};

#endif
```

myEditor.cpp

```
#include "myEditor.h"
#include <QMouseEvent>
MyEditor::MyEditor(QWidget *parent) : QWidget(parent), shapeCount(0),
currentShape(nullptr), isDragging(false)
{};



MyEditor::~MyEditor() {
   delete currentShape;
   for (int i = 0; i < shapeCount; i++) {
      delete pcshape[i];
   }
};

void MyEditor::setCurrentShape(Shape *shape) {
   delete currentShape;
   currentShape = shape;
}
```

```
void MyEditor::clearCanvas() {
    for (int i = 0; i < shapeCount; i++) {
        delete pcshape[i];
        pcshape[i] = nullptr;
    }
    shapeCount = 0;
    redrawCanvas();
}

void MyEditor::redrawCanvas() {
    pixmap.fill(Qt::white);
    QPainter painter(&pixmap);

    for (int i = 0; i < shapeCount; i++)
    {
        if (pcshape[i])
        {
            pcshape[i]->show(painter);
        }
    }
    update();
}

void MyEditor::mousePressEvent(QMouseEvent *event) {

    if (event->button() == Qt::LeftButton && currentShape != nullptr)
```

```
    {
        if (shapeCount >= N)
            return;

        startPt = event->pos();
        endPt = event->pos();
        isDragging = true;

        if (typeid(*currentShape) == typeid(PointShape))
        {
            pcshape[shapeCount] = currentShape->clone();
            pcshape[shapeCount]->set(event->pos().x(), event->pos().y(),
event->pos().x(), event->pos().y());
            shapeCount++;
            isDragging = false;
            redrawCanvas();
        }
    }
}

void MyEditor::mouseMoveEvent(QMouseEvent *event)
{
    if (isDragging)
    {
        endPt = event->pos();
        update();
    }
```

```
}

void MyEditor::mouseReleaseEvent(QMouseEvent *event)
{
   if (event->button() == Qt::LeftButton && isDragging)
   {
      isDragging = false;
      endPt = event->pos();

      if (currentShape && shapeCount < N)
      {
         pcshape[shapeCount] = currentShape->clone();
         pcshape[shapeCount]->set(startPt.x(), startPt.y(), endPt.x(), endPt.y());
         shapeCount++;
         redrawCanvas();
      }
   }
}

void MyEditor::resizeEvent(QResizeEvent *event)
{
   QWidget::resizeEvent(event);
   pixmap = QPixmap(size());
   redrawCanvas();
}
```

```
void MyEditor::paintEvent(QPaintEvent *event)
{
   Q_UNUSED(event);
   QPainter painter(this);

   painter.drawPixmap(0, 0, pixmap);

   if (isDragging && currentShape)
   {
      currentShape->set(startPt.x(), startPt.y(), endPt.x(), endPt.y());
      currentShape->rubber(painter);
   }
}
```

myEditor.h

```
#ifndef MYEDITOR_H
#define MYEDITOR_H

#include <QWidget>
#include <QPixmap>
#include <QPoint>
#include "shape.h"

class MyEditor : public QWidget
{
   Q_OBJECT
```

```
public:
    MyEditor(QWidget *parent = nullptr);
    ~MyEditor();

    void setCurrentShape(Shape *shape);
    void clearCanvas();

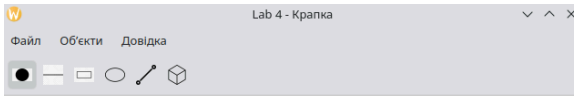
protected:
    void paintEvent(QPaintEvent *event) override;
    void mousePressEvent(QMouseEvent *event) override;
    void mouseMoveEvent(QMouseEvent *event) override;
    void mouseReleaseEvent(QMouseEvent *event) override;
    void redrawCanvas();
    void resizeEvent(QResizeEvent *event) override;

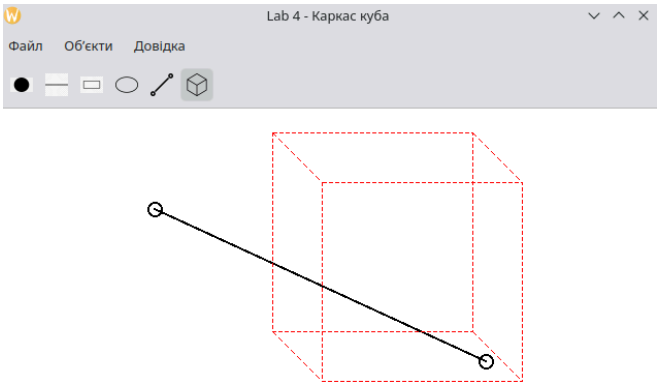
    bool isDragging;
    QPoint startPt;
    QPoint endPt;

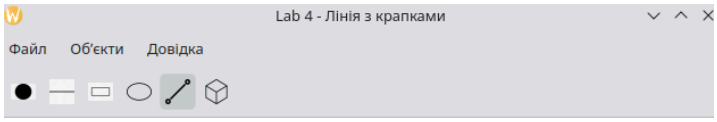
    static const int N = 117;
    Shape *currentShape;
    Shape *pcshape[N] = {nullptr};
    int shapeCount;
    QPixmap pixmap;
};

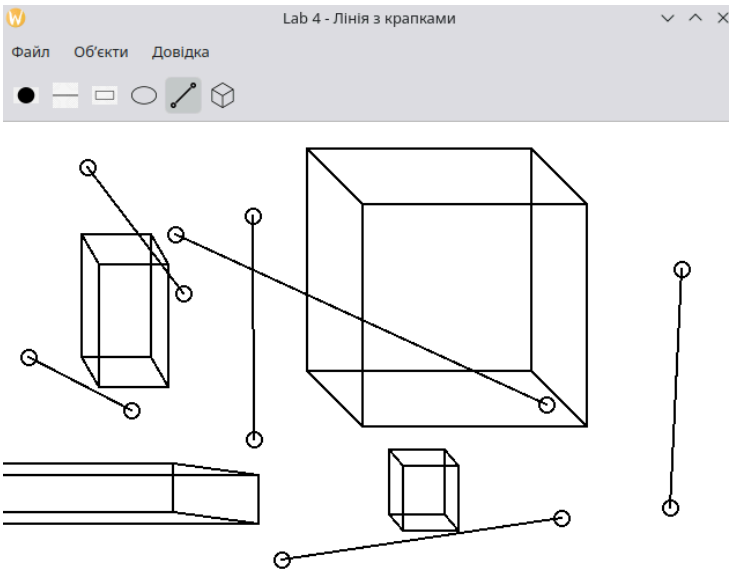
#endif
```

Скріншоти

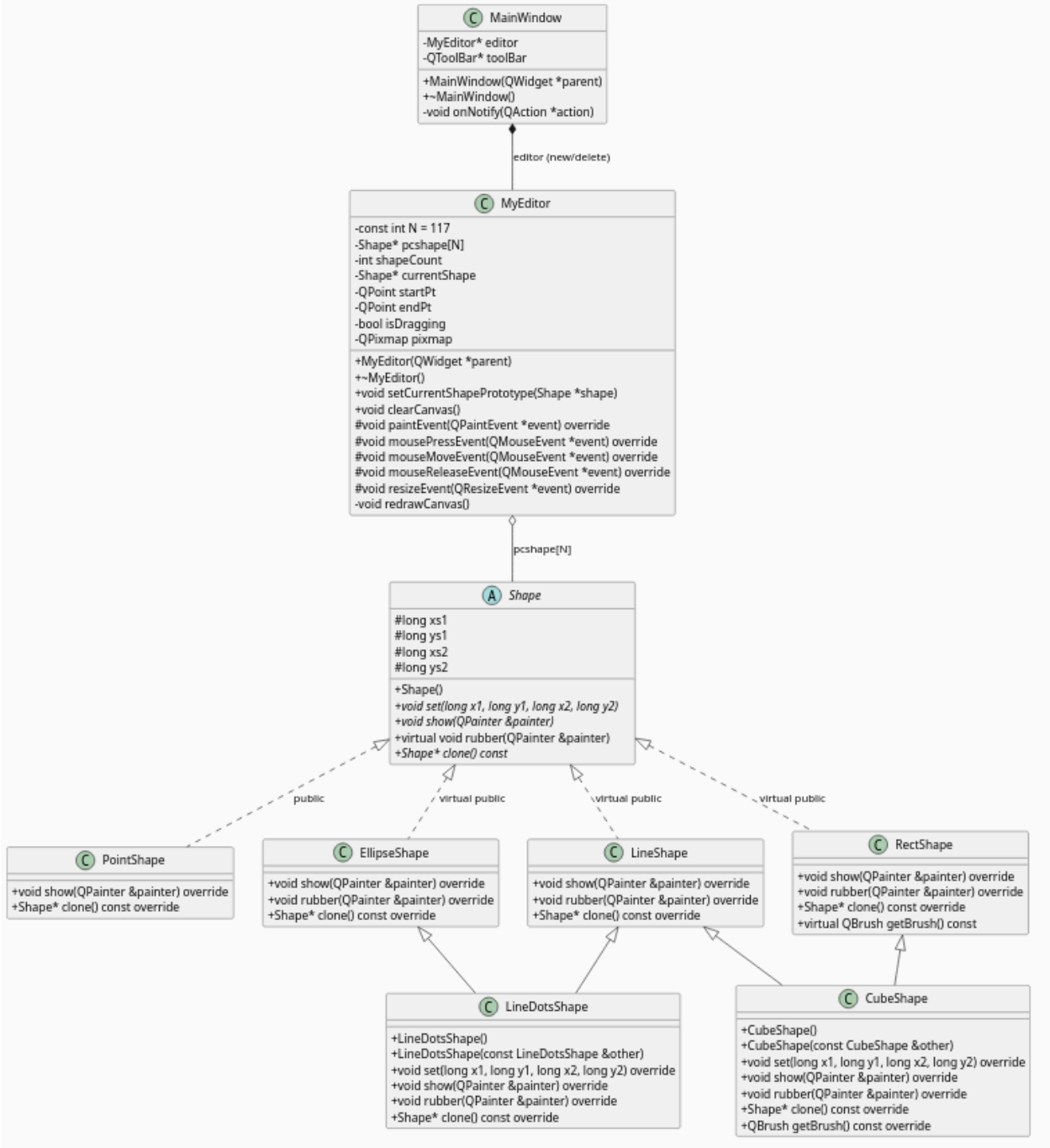

Lab 4 - Крапка
Файл
Об'єкти
Довідка


Lab 4 - Каркас куба
Файл
Об'єкти
Довідка


Lab 4 - Лінія з крапками
Файл
Об'єкти
Довідка


Lab 4 - Лінія з крапками
Файл
Об'єкти
Довідка

Діаграма залежностей класів



Висновки:

У ході виконання лабораторної роботи 4 було успішно розроблено графічний редактор на базі фреймворку Qt, що реалізує динамічне створення об'єктів та об'єктно-орієнтовану ієрархію геометричних фігур. Завдяки застосуванню множинного та віртуального успадкування було реалізовано складні фігури (лінію з еліпсами та каркас куба) без витоків пам'яті та конфліктів у стані.